



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА № 6
з дисципліни «Технології розроблення програмного забезпечення»
Тема: «Патерни проектування»

Виконала:

Студентка групи ІА-31

Соколова Поліна

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

15. Е-mail клієнт (singleton, builder, decorator, template method, interpreter, client-server)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Хід роботи

Короткі теоретичні відомості:

1. Abstract Factory

Призначення: Створення сімейств пов'язаних об'єктів без вказівки конкретних класів.

Коли використовувати:

- Система не повинна залежати від способу створення об'єктів
- Потрібно працювати з різними сімействами продуктів
- Необхідно забезпечити сумісність об'єктів у межах одного сімейства

Переваги:

- Гарантує сумісність створених об'єктів
- Спрощує заміну сімейств продуктів
- Відокремлює створення від використання

Недоліки:

- Складність додавання нових типів продуктів
- Збільшення кількості класів

2. Factory Method

Призначення: Визначає інтерфейс для створення об'єктів, дозволяючи підкласам вирішувати, які класи інстанціювати.

Коли використовувати:

- Заздалегідь невідомі типи об'єктів
- Потрібна можливість розширення способів створення
- Делегування створення підкласам

Переваги:

- Усуває залежність від конкретних класів
- Спрощує додавання нових типів продуктів
- Централізує код створення об'єктів

Недоліки:

- Може створити паралельні ієрархії класів

3. Memento

Призначення: Збереження та відновлення попереднього стану об'єкта без порушення інкапсуляції.

Коли використовувати:

- Потрібна функція "відміни" операцій
- Необхідно зберігати історію станів
- Пряме отримання стану порушує інкапсуляцію

Переваги:

- Не порушує інкапсуляцію
- Спрощує структуру Originator
- Легко реалізувати відкат змін

Недоліки:

- Великі витрати пам'яті при частих збереженнях
- Потрібно керувати життєвим циклом знімків

4. Observer

Призначення: Визначає залежність "один-до-багатьох", коли зміна стану одного об'єкта автоматично оповіщає всіх залежних.

Коли використовувати:

- Зміна одного об'єкта впливає на інші
- Об'єкт повинен сповіщати інші без тісного зв'язку
- Потрібна система підписки/розсилки

Переваги:

- Слабкий зв'язок між об'єктами
- Динамічне додавання/видалення спостерігачів
- Можливість асинхронної обробки

Недоліки:

- Відсутність гарантій послідовності сповіщень
- Можливі проблеми з продуктивністю при великій кількості спостерігачів

5. Decorator

Призначення: Динамічне додавання нової функціональності об'єктам без зміни їх структури.

Коли використовувати:

- Потрібно додавати обов'язки об'єктам динамічно
- Розширення через спадкування непрактичне
- Необхідно комбінувати різні поведінки

Переваги:

- Гнучкіше за спадкування

- Додавання функцій "на льоту"
- Дрібні спеціалізовані класи

Недоліки:

- Велика кількість маленьких класів
- Складність при багатьох рівнях декорування
- Важко відстежити порядок застосування декораторів

Шаблон «Decorator»

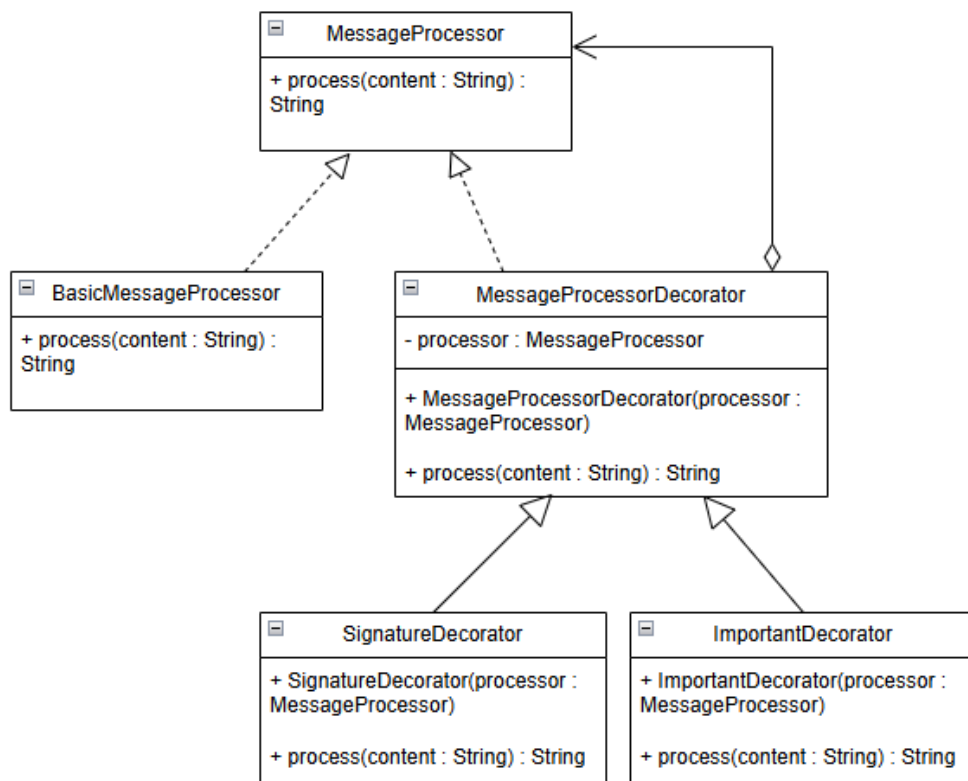


Рис.1 Діаграма класів, яка представляє використання шаблону

На діаграмі показано реалізацію шаблону Decorator, що розширює поведінку об'єкта `MessageProcessor` без зміни його структури. Абстрактний декоратор `MessageProcessorDecorator` агрегує компонент `MessageProcessor` та делегує йому основну поведінку. Конкретні декоратори (`SignatureDecorator`, `ImportantDecorator`) успадковують базовий декоратор і додають нові операції. Основний компонент `BasicMessageProcessor` реалізує інтерфейс `MessageProcessor` та забезпечує базову логіку обробки повідомлення.

```

1 public interface MessageProcessor { 8 usages 5 implementations
    String process(String content); 5 usages 5 implementations
2 }

public abstract class MessageProcessorDecorator implements MessageProcessor { 3 usages 3 inheritors
    protected final MessageProcessor processor; 2 usages

    protected MessageProcessorDecorator(MessageProcessor processor) { 3 usages
        this.processor = processor;
    }

    @Override 5 usages 3 overrides
    public String process(String content) {
        return processor.process(content);
    }
2 }

public class BasicMessageProcessor implements MessageProcessor { 1 usage
    @Override 5 usages
    public String process(String content) {
        return content;
    }
3 }

public class ImportantDecorator extends MessageProcessorDecorator { 1 usage

    public ImportantDecorator(MessageProcessor processor) { 1 usage
        super(processor);
    }

    @Override 5 usages
    public String process(String content) {
        String base = super.process(content);
        return "[IMPORTANT]\n" + base;
    }
4 }

public class SignatureDecorator extends MessageProcessorDecorator { 1 usage

    public SignatureDecorator(MessageProcessor processor) { 1 usage
        super(processor);
    }

    @Override 5 usages
    public String process(String content) {
        String base = super.process(content);
        return base + "\n-- Sent by EmailClient";
    }
5 }

```

Рис.2 Фрагменти коду по реалізації шаблону

(1– interface MessageProcessor, 2 – class MessageProcessorDecorator, 3 – class BasicMessageProcessor, 4 – class ImportantDecorator , 5 – class SignatureDecorator)

Шаблон застосовано до процесу підготовки тексту повідомлення перед його збереженням або надсиланням. Для цього було введено інтерфейс `MessageProcessor`, який визначає єдиний метод `process()`. Клас `BasicMessageProcessor` є базовою реалізацією, що виступає стандартним компонентом без додаткової логіки.

Було створено абстрактний декоратор `MessageProcessorDecorator`, який також реалізує інтерфейс `MessageProcessor`, але додатково містить поле типу `MessageProcessor`. Це дає змогу обгорнути базовий об'єкт у нові рівні поведінки.

Створені конкретні декоратори — `SignatureDecorator` та `ImportantDecorator` — успадковують абстрактний клас-декоратор та розширюють його функціональність: додавання підпису, маркування повідомлення як важливого.

У контролері повідомлень `MessageController` було змінено логіку створення та обробки листа: перед відправленням текст повідомлення тепер проходить через ланцюжок декораторів, що дозволяє послідовно додавати нову поведінку, не змінюючи базову реалізацію класу `Message`.

Висновок: у даній лабораторній роботі було реалізовано шаблон «Decorator», який забезпечив можливість динамічного розширення функціональності обробки повідомлень у системі без зміни структури їх базових класів. Було розроблено інтерфейс компонента, базовий процесор повідомлень, абстрактний декоратор та кілька конкретних декораторів, що додають додаткові можливості.

Відповіді на питання:

1. Яке призначення шаблону «Абстрактна фабрика»?

Шаблон використовується для створення сімейств об'єктів без вказівки їх конкретних класів. Він структурує знання про схожі об'єкти та створює можливість взаємозаміни різних сімейств продуктів.

3. Які класи входять в шаблон «Абстрактна фабрика», та яка між ними взаємодія?

AbstractFactory – загальний інтерфейс фабрики з методами створення продуктів
ConcreteFactory (наприклад, HighTechFabric, ModernFabric) – конкретні реалізації фабрик.

AbstractProduct – інтерфейси продуктів (ITable, IChair тощо)

ConcreteProduct – конкретні реалізації продуктів (HighTechTable, ModernTable)

Client – використовує інтерфейси фабрики та продуктів

Взаємодія: клієнт працює через інтерфейс AbstractFactory, який створює сімейства узгоджених продуктів одного стилю.

4. Яке призначення шаблону «Фабричний метод»?

Шаблон визначає інтерфейс для створення об'єктів певного базового типу, дозволяючи підкласам змінювати тип створюваних об'єктів. Це "віртуальний конструктор" для впровадження власних конструкторів об'єктів.

6. Які класи входять в шаблон «Фабричний метод», та яка між ними взаємодія?

Creator (PacketCreator) – базовий клас з фабричним методом

ConcreteCreator (TcpCreator, UdpCreator) – конкретні створювачі

Product (Packet) – базовий тип продукту

ConcreteProduct (TcpPacket, UdpPacket) – конкретні продукти

Взаємодія: базовий клас визначає фабричний метод, а підкласи перевизначають його для створення різних типів продуктів, зберігаючи базову функціональність.

7. Чим відрізняється «Абстрактна фабрика» від «Фабричний метод»?

Абстрактна фабрика створює сімейства узгоджених об'єктів різних типів.

Фабричний метод створює об'єкти одного типу, але різних підтипів.

Абстрактна фабрика використовує композицію, Фабричний метод – успадкування.

8. Яке призначення шаблону «Знімок» (Memento)?

Використовується для збереження і відновлення стану об'єктів без порушення інкапсуляції. Дозволяє зберігати історію станів та відкочувати зміни.

10. Які класи входять в шаблон «Знімок», та яка між ними взаємодія?

Originator – початковий об'єкт, що зберігає та відновлює свій стан

Memento – об'єкт для збереження стану Originator

Caretaker – зберігає мemento об'єкти та керує ними

Взаємодія: Originator створює Memento зі своїм станом; Caretaker зберігає Memento; лише Originator може отримати стан з Memento.

11. Яке призначення шаблону «Декоратор»?

Призначений для динамічного додавання функціональних можливостей об'єкту під час роботи програми. "Обертає" об'єкт зі збереженням його функцій, додаючи нову поведінку.

13. Які класи входять в шаблон «Декоратор», та яка між ними взаємодія?

Component – базовий інтерфейс компонента

ConcreteComponent – конкретний компонент

Decorator – базовий декоратор, що містить Component

ConcreteDecorator – конкретні декоратори з додатковою функціональністю

Взаємодія: декоратор агрегує компонент та делегує йому виклики, додаючи власну поведінку до/після виклику.

14. Які є обмеження використання шаблону «Декоратор»?

Велика кількість дрібних класів може ускладнити розуміння структури коду.

Важко конфігурувати об'єкти, загорнуті в декілька обгортки одночасно.